



Pronóstico de Contaminantes Atmosféricos ($PM_{2.5}$, O_3 y NO_x) mediante la Metodología Box–Jenkins y Modelos de Aprendizaje Automático



Abraham Alvarez Méndez¹, Nereyda Wiwiema De la Cruz Martínez², Facundo Juárez Santiago³, Adriana Lara López⁴

¹ Centro Universitario UAEM Valle de México, Av. Hermenegildo Galeana S/N, Col. El Potrero, C.P. 52975, Atizapán de Zaragoza, Estado de México, México.

¹mendezabraham701alm@gmail.com

² Universidad Autónoma de Nayarit, Ciudad de la Cultura Amado Nervo, Av. de la Cultura S/N, C.P. 63155, Tepic, Nayarit, México.

²22009960@uan.edu.mx

³ Universidad Autónoma de Coahuila, Blvd. Venustiano Carranza S/N, Col. República, C.P. 25280, Saltillo, Coahuila, México.

³sfacundo@uadec.edu.mx

⁴ Av. Instituto Politécnico Nacional, S/N, Ud. Profesional Adolfo López Mateos, Edif. 9, Col. Nueva Industrial Vallejo, San Pedro Zacatenco., Gustavo A. Madero, C.P. 07700, Ciudad de México.

⁴alaral@ipn.mx

Resumen: Este trabajo compara dos paradigmas de modelado para pronosticar tres contaminantes atmosféricos críticos para la calidad del aire de la Ciudad de México: partículas suspendidas $PM_{2.5}$, ozono (O_3) y óxidos de nitrógeno (NO_x), usando series diarias y semanales de la estación Merced. Por un lado, se aplica la metodología clásica de Box–Jenkins para ajustar modelos ARMA/ARIMA y SARIMA estacionales. Por otro lado, se exploran dos modelos de aprendizaje automático no lineales basados en árboles, XGBoost y Random Forest, utilizando variables rezagadas, diferencias, medias móviles y tendencia local como predictores. Dado que las series originales presentan datos faltantes, se evaluaron previamente tres técnicas de imputation: relleno hacia adelante/atrás, interpolación lineal y k-vecinos más cercanos (KNN), seleccionando la más adecuada mediante un experimento de eliminación artificial de datos conocidos. Los resultados muestran que la imputación por KNN produce los menores errores de reconstrucción y que XGBoost reduce de forma notable el error de pronóstico para NO_x y O_3 frente a los modelos lineales clásicos. Asimismo, Random Forest identifica a la presión atmosférica como la variable más predictiva para $PM_{2.5}$ (71–74 % de la importancia total), un hallazgo relevante para la interpretación física del fenómeno que los modelos univariados de Box–Jenkins no pueden ofrecer.

Palabras Clave: *Box–Jenkins, calidad del aire, imputación de datos, KNN, Random Forest, XGBoost*

Abstract: This paper compares two modeling paradigms to forecast three critical air pollutants in Mexico City: fine particulate matter ($PM_{2.5}$), tropospheric ozone (O_3), and nitrogen oxides

(NO_x), using daily and weekly series from the Merced monitoring station. On one hand, the classical Box–Jenkins methodology is applied to fit ARMA/ARIMA and seasonal SARIMA models. On the other hand, two non-linear tree-based machine learning models, XGBoost and Random Forest, are explored using lagged variables, differences, moving averages, and local trends as predictors. Given that the original series contain missing values, three imputation techniques were previously evaluated: forward/backward fill, linear interpolation, and k-nearest neighbors (KNN), selecting the most appropriate one through an artificial data elimination experiment. The results show that KNN imputation yields the lowest reconstruction errors and that XGBoost significantly reduces the forecasting error for NO_x and O₃ compared to classical linear models. Furthermore, Random Forest identifies atmospheric pressure as the most predictive variable for PM_{2.5} (71–74% of total importance), a key finding for the physical interpretation of the phenomenon that univariate Box–Jenkins models cannot provide.

Keywords: *air quality, Box–Jenkins, data imputation, KNN, Random Forest, XGBoost*

INTRODUCCIÓN

La contaminación atmosférica es uno de los principales problemas de salud pública en zonas urbanas densamente pobladas como la Ciudad de México. El monitoreo continuo de contaminantes como el material particulado fino (PM_{2.5}), el ozono troposférico (O₃) y los óxidos de nitrógeno (NO_x) permite anticipar episodios de mala calidad del aire y activar medidas de mitigación. Sin embargo, las estaciones de monitoreo frecuentemente presentan huecos en sus registros debido a fallas de instrumentación, mantenimiento o errores de transmisión, lo cual complica el uso directo de técnicas estadísticas clásicas de series de tiempo.

El presente proyecto tiene tres objetivos entrelazados:

1. Reconstruir las series diarias y semanales de doce variables ambientales mediante tres estrategias de imputación de datos faltantes y determinar cuál conserva mejor la estructura estadística original.
2. Modelar y pronosticar las concentraciones futuras aplicando la metodología Box–Jenkins, evaluando el desempeño del pronóstico a

cuatro pasos mediante métricas de error estándar.

3. Explorar modelos de aprendizaje automático no lineales (XGBoost y Random Forest) utilizando rezagos de los propios contaminantes y el resto de las variables ambientales como predictores, y comparar su desempeño contra los modelos Box–Jenkins clásicos.

TRABAJO RELACIONADO

El análisis y pronóstico de contaminantes atmosféricos ha ganado un interés significativo debido a su impacto directo en la salud pública y el medio ambiente. En la literatura reciente, la modelación de la calidad del aire abarca desde enfoques estocásticos tradicionales hasta algoritmos complejos de aprendizaje automático (*Machine Learning*) y aprendizaje profundo (*Deep Learning*).

Un aspecto crítico en el tratamiento de datos ambientales es la presencia de valores faltantes ocasionados por fallas en la transmisión de datos, calibración de sensores o mantenimientos de las estaciones de monitoreo. Diversos estudios han

demostrado que la elección del método de imputación impacta de manera directa en el desempeño de los modelos de predicción subsiguientes. En particular, [?] llevaron a cabo un análisis exhaustivo evaluando múltiples técnicas de imputación (incluyendo métodos estadísticos simples y algoritmos basados en vecinos más cercanos) combinados con modelos de pronóstico para datos de $PM_{2.5}$ en la región de Delhi. Sus hallazgos subrayan la importancia de reconstruir adecuadamente la continuidad temporal antes de proceder a la fase de modelación.

Siguiendo esta línea metodológica, nuestro trabajo evalúa de manera comparativa cuatro enfoques de imputación de datos (Relleno hacia adelante, Relleno hacia atrás, Interpolación Lineal y K -NN Imputer) aplicados no solo al material particulado ($PM_{2.5}$), sino también a otros contaminantes atmosféricos fundamentales como el ozono (O_3) y los óxidos de nitrógeno (NO_x).

I METODOLOGÍA

A ¿Qué es una serie de tiempo?

Una serie de tiempo es un conjunto de observaciones cronológicas generadas de manera secuencial en el tiempo.

B La metodología Box–Jenkins

Definición 1. La metodología Box–Jenkins consiste en extraer los movimientos predecibles de los datos observados y separarlos de la parte no predecible o completamente aleatoria (ruido blanco). A partir de dicha separación se construyen modelos autorregresivos de medias móviles (ARMA) y sus extensiones integradas y estacionales (ARIMA/SARIMA) que permiten generar pronósticos de corto plazo con intervalos de confianza.[1]

La metodología se ejecuta en cinco etapas

iterativas:[1]

1. **Análisis inicial (estacionariedad y estacionalidad):** Se aplica la prueba de Dickey–Fuller Aumentada (ADF) para verificar la estacionariedad.
2. **Identificación:** Se combina la inspección visual de los correlogramas con algoritmos de búsqueda automática usando los criterios AIC, BIC y HQ.
3. **Estimación de parámetros:** El modelo tentativo se reestima para obtener los coeficientes definitivos y su significancia estadística.
4. **Diagnóstico del modelo:** Se prueban las hipótesis de normalidad (Shapiro–Wilk), no autocorrelación (Ljung–Box) y estacionariedad sobre los residuos.
5. **Pronóstico:** Se genera el pronóstico puntual acotado a 4 o 5 pasos hacia adelante, ya que el margen de error crece en horizontes lejanos.

C ¿Qué es Random Forest?

Random Forest (RF) es un algoritmo de aprendizaje automático basado en el método de ensamblado (*ensemble learning*), propuesto por Breiman en 2001 [6]. El algoritmo construye un conjunto de árboles de decisión independientes utilizando muestras obtenidas mediante remuestreo con reemplazo (*bootstrap*). Además, durante la construcción de cada árbol, en cada nodo se selecciona aleatoriamente un subconjunto de variables predictoras para determinar la mejor división posible. Esta estrategia disminuye la correlación entre los árboles individuales, reduce el riesgo de sobreajuste y mejora la capacidad de generalización del modelo.

D Ventanas deslizantes de Random Forest

Con el propósito de incorporar la dependencia temporal de las series de tiempo, se implementó una estrategia de ventanas deslizantes (*sliding windows*) para generar nuevas variables predictoras a partir de los valores históricos de cada contaminante y de las variables meteorológicas. Para la serie diaria, las características generadas fueron las siguientes:

- Rezagos (*lags*) de 1, 3, 7 y 14 días.
- Diferencias temporales de 1 y 7 días.
- Medias móviles calculadas sobre ventanas de 7 y 14 días.
- Tendencia local estimada mediante una regresión lineal sobre una ventana de 7 observaciones.

Para la serie semanal se empleó el mismo procedimiento; sin embargo, las ventanas fueron adaptadas a la resolución temporal de los datos:

- Rezagos de 1, 2, 4 y 8 semanas.
- Diferencias temporales de 1 y 4 semanas.
- Medias móviles de 4 y 8 semanas.
- Tendencia local calculada mediante una regresión lineal sobre una ventana de 4 observaciones.

Las variables derivadas fueron calculadas para cada uno de los contaminantes atmosféricos y variables meteorológicas consideradas en el estudio. Finalmente, las observaciones con valores faltantes generados por el cálculo de las ventanas fueron eliminadas antes del entrenamiento del modelo.

D.1 Hiperparámetros del modelo Random Forest

El modelo de regresión Random Forest fue implementado utilizando la biblioteca **Scikit-learn**. Los principales hiperparámetros utilizados durante el entrenamiento fueron los siguientes:

1. **n_estimators:** Número de árboles de decisión que conforman el bosque aleatorio. Un mayor número de árboles suele producir predicciones más estables, aunque incrementa el tiempo de entrenamiento.
2. **max_depth:** Profundidad máxima permitida para cada árbol de decisión. Este parámetro controla la complejidad del modelo y su capacidad para ajustarse a los datos.
3. **min_samples_leaf:** Número mínimo de muestras requeridas para formar un nodo hoja. Su propósito es evitar que los árboles generen particiones con muy pocas observaciones, reduciendo así el sobreajuste.
4. **n_jobs:** Número de núcleos del procesador utilizados durante el entrenamiento del modelo. Permite paralelizar la construcción de los árboles para disminuir el tiempo de ejecución.
5. **random_state:** Semilla empleada para la generación de números aleatorios durante el entrenamiento, garantizando la reproducibilidad de los resultados.

E ¿Qué es XGBoost?

XGBoost (*Extreme Gradient Boosting*) es una biblioteca de *Machine Learning* distribuida y de código abierto, creada por Tianqi Chen, que implementa árboles de decisión potenciados por gradiente mediante el algoritmo de descenso de gradiente [16]. Se caracteriza por su velocidad, efi-

ciencia computacional y capacidad de escalar adecuadamente en conjuntos de datos de gran tamaño [16]. Funciona como un sistema de árboles de decisión en equipo, donde cada árbol aprende de los errores del anterior para mejorar el resultado final [17]. Su gran ventaja frente a otros métodos similares es que incluye en el modelo la regularización. Esto evita que el modelo se memorice los datos de entrenamiento de forma exacta (sobreajuste), logrando que sea mucho más preciso y confiable cuando analiza datos totalmente nuevos.

F Hiperparámetros de XGBoost

Los principales hiperparámetros de un modelo de *boosting* como XGBoost son los siguientes [17, 18]:

1. **n_estimators:** Número de árboles del ensamble; más árboles implican mayor complejidad y riesgo de sobreajuste.
2. **max_depth:** Profundidad máxima de cada árbol; valores altos aumentan las interacciones entre variables y el sobreajuste.
3. **learning_rate:** Tasa de aprendizaje (*shrinkage*) que reduce el impacto de cada árbol; valores bajos requieren más árboles pero generalizan mejor.
4. **subsample:** Fracción de muestras usadas para entrenar cada árbol, previniendo el sobreajuste mediante aleatoriedad.
5. **colsample_bytree:** Fracción de variables usadas en cada árbol, con el mismo efecto regularizador que **subsample**.
6. **min_child_weight:** Peso mínimo de las observaciones en un nodo hijo; valores altos generan árboles más simples.
7. **gamma:** Reducción mínima de pérdida necesaria para dividir un nodo; valores altos vuelven al modelo más conservador.

8. **reg_alpha:** Regularización L1 (Lasso), que puede llevar coeficientes a cero y seleccionar variables relevantes.

9. **reg_lambda:** Regularización L2 (Ridge), que penaliza coeficientes grandes y aporta mayor estabilidad.

II METODOS Y MATERIALES

A Búsqueda Datos

Los datos fueron obtenidos del sitio web de la Dirección de Monitoreo Atmosférico de la Ciudad de México, correspondientes al periodo 2018-2023. La información proviene de tres bases de datos: la Red Automática de Monitoreo Atmosférico (RAMA), que aporta los contaminantes **o3**, **no2**, **nox**, **so2**, **co**, **pm10** y **pm25**; la Red de Meteorología y Radiación Solar (REDMET), que proporciona los parámetros meteorológicos temperatura (**tmp**), humedad relativa (**rh**), dirección del viento (**wdr**) y velocidad del viento (**wsp**); y la base de datos de presión atmosférica (**pa**), también perteneciente a REDMET. En total se integraron doce variables ambientales, de las cuales **o3**, **pm25** y **nox** constituyen las variables de mayor interés para el pronóstico.

B Creación y limpieza de los datos

Se descargaron 18 carpetas en formato **.zip**, correspondientes a los 6 años y las 3 bases de datos mencionadas, cuya resolución temporal original es horaria. La importación de dichas carpetas se realizó en Python para optimizar el tiempo de procesamiento. Posteriormente se identificó qué estaciones contaban con las 12 variables seleccionadas y cuál de ellas presentaba el menor número de valores faltantes, seleccionando así la estación Merced como la óptima. Sobre esta base se aplicó un primer filtro para conservar

únicamente el intervalo horario de 6:00 a 22:00, y un segundo filtro mediante el cual el promedio diario de cada variable solo se calcula si se cuenta con al menos 11 horas de registro en ese día; de lo contrario, el día se descarta. Aun con estos filtros, persisten algunos valores faltantes en la base resultante.

C Análisis de los datos faltantes

Se generó una gráfica para cuantificar los días faltantes por variable y así definir la estrategia de tratamiento a seguir. Asimismo, se detectaron valores atípicos (*outliers*) a lo largo de las series de tiempo; sin embargo, se optó por conservarlos sin eliminarlos ni modificarlos, dado que en un análisis de series de tiempo la eliminación de registros puede romper la continuidad temporal y distorsionar el comportamiento real de la serie.

D Técnicas de imputación

Se trabajó con tres técnicas de imputación de datos faltantes.

1. **Interpolación lineal:** Es una técnica monovariante diseñada para datos secuenciales o series temporales que calcula el valor faltante trazando una línea recta entre los puntos conocidos inmediatamente anterior y posterior al vacío [9].
2. **Relleno hacia adelante y hacia atrás (FFill/BFill):** Métodos heurísticos y de bajo costo computacional que propagan datos válidos para rellenar vacíos: FFill arrastra el último valor conocido hacia adelante, mientras que BFill copia el primer valor posterior hacia atrás [10].
3. **K-Nearest Neighbors (KNN):** Este método multivariante estima los datos faltantes identificando los k registros más parecidos (vecinos) en el espacio de variables

mediante métricas de distancia, como la euclidiana [8].

E Métricas de evaluación

El rendimiento de los modelos, tanto en tareas de pronóstico como en la imputación de datos faltantes, se evaluó utilizando las siguientes métricas.

1. **Error absoluto medio (MAE):** Cuantifica el promedio de las diferencias absolutas entre los valores predichos y los valores reales, ofreciendo una medida del error en las mismas unidades que la variable analizada [12].
2. **Raíz del error cuadrático medio (RMSE):** Calcula la raíz cuadrada del promedio de los errores al cuadrado, penalizando en mayor medida las desviaciones grandes entre el valor predicho y el valor real [11].
3. **Coefficiente de determinación (R^2):** Indica la proporción de la varianza de la variable dependiente que es explicada por el modelo, permitiendo evaluar la calidad del ajuste [12].

La selección de estas métricas facilita una comparación exhaustiva y detallada entre diferentes tipos de modelos, permitiendo medir con precisión su rendimiento predictivo.

F Técnicas de escalamiento y normalización

Estas técnicas resultan útiles porque permiten a los algoritmos de *Machine Learning* mejorar el ajuste de los distintos modelos establecidos.

RobustScaler es un escalador lineal que ajusta las variables numéricas utilizando la mediana y el rango intercuartílico (IQR) en lugar de

la media y la desviación estándar, lo que evita que los valores atípicos (*outliers*) distorsionen la escala de los datos sin alterar la forma de su distribución original [14].

PowerTransformer es una transformación no lineal (como Box-Cox o Yeo-Johnson) diseñada específicamente para estabilizar la varianza y modificar activamente la distribución de los datos para que se asemeje lo más posible a una distribución normal (gaussiana) [13].

G Validación cruzada de series temporales para la selección de los parámetros

La validación cruzada de series temporales se basó en una ventana expansiva que respeta rigurosamente el orden cronológico. En este enfoque, los datos se fragmentan secuencialmente en lugar de forma aleatoria: el conjunto de entrenamiento se amplía en cada etapa para incorporar registros más recientes, mientras que el modelo se evalúa en el bloque de tiempo inmediatamente posterior. Este proceso se repite de manera iterativa, desplazando el punto de corte hacia el futuro [15].

H ¿Qué es RandomizedSearchCV?

Para optimizar el modelo de XGBoost se empleó **RandomizedSearchCV**, una técnica que explora de manera aleatoria un número n de combinaciones (**n_iter**) dentro del espacio de búsqueda de los hiperparámetros descritos en la Sección anterior, con el objetivo de encontrar la configuración que minimice el error del modelo [19].

La búsqueda se configuró con los siguientes criterios:

1. **Validación cruzada temporal (cv=tscv):** Cada combinación se evalúa respetando el orden cronológico de los datos, conforme al esquema descrito en la Sección de validación cruzada.

2. **Métrica de evaluación:** Se utiliza el RMSE negativo, ya que **scikit-learn** está diseñado para maximizar métricas; negar el RMSE equivale, por lo tanto, a minimizarlo.

3. **Reproducibilidad (random_state=42):** Se fija una semilla para garantizar que los resultados sean replicables.

4. **Paralelización (n_jobs=-1):** Se aprovechan todos los núcleos disponibles del procesador para acelerar el proceso de búsqueda.

I Evaluación de los métodos de imputación

Para elegir la técnica más confiable se anularon artificialmente valores conocidos y se calculó el error de reconstrucción. La Tabla 1 resume los resultados.

Partícula (PM _{2.5})			
Técnica	MAE	RMSE	RMSLE
KNN	5.6230	7.1059	0.3597
Interpolación lineal	9.4879	11.5376	0.5867
Relleno (bfill/ffill)	10.5176	12.5647	0.6331
Óxidos de nitrógeno (NO _x)			
Técnica	MAE	RMSE	RMSLE
KNN	6.9502	10.2162	0.2542
Interpolación lineal	18.6759	24.9435	0.6761
Relleno (bfill/ffill)	23.6131	29.9142	0.8175
Ozono (O ₃)			
Técnica	MAE	RMSE	RMSLE
KNN	11.5482	14.8941	0.4172
Interpolación lineal	26.8128	31.9324	0.9995
Relleno (bfill/ffill)	33.5557	39.1766	1.2955
Promedio de las 12 variables			
Técnica	MAE	RMSE	RMSLE
KNN	12.7780	15.4333	0.2461
Interpolación lineal	17.9207	22.0690	0.4472
Relleno (bfill/ffill)	20.7992	25.1600	0.5399

Tabla 1: Comparación de métricas de imputación por contaminante y técnica.

Conclusión del análisis de imputación:

En los tres contaminantes analizados, la técnica KNN obtiene consistentemente el menor error, ya que aprovecha la estructura correlacionada de las variables meteorológicas concurrentes.

J Matriz de Correlación

Se calculó la matriz de correlación entre las variables de calidad del aire y meteorológicas (Figura ??). Se observan correlaciones fuertes y positivas entre los contaminantes primarios, destacando PM_{2.5}-PM₁₀ ($r = 0.86$) y NO_x-NO₂ ($r = 0.89$), lo que sugiere fuentes de emisión comunes. Por su parte, las variables meteorológicas muestran relaciones inversas con los contaminantes, en particular la velocidad del viento con NO₂ ($r = -0.60$), reflejando el efecto dispersivo del viento, mientras que la temperatura se correlaciona positivamente con el ozono ($r = 0.59$).

III MODELADO BOX-JENKINS POR CONTAMINANTE

Para cada contaminante se ajustaron modelos ARMA/ARIMA (diarios) y SARIMA (semanales, $m = 52$), reservando los últimos 4 pasos como conjunto de prueba fuera de muestra.

IV DIAGNÓSTICO DE LOS MODELOS

En todos los casos se verificaron los supuestos exigidos por la metodología Box-Jenkins sobre los residuos del modelo final. En primer lugar, se aplicó la **Prueba de Dickey-Fuller Aumentada (ADF)** sobre los residuos, bajo la hipótesis nula (H_0) de que estos no son estacionarios, frente a la alternativa (H_1) de que sí lo son; en los modelos reportados se obtuvo un p -valor ≈ 0.0000 , por

lo que se rechaza H_0 y se concluye que los residuales se comportan como ruido blanco, validando la adecuación del modelo. En segundo lugar, mediante la **Prueba de Shapiro-Wilk** se evaluó el supuesto de normalidad en la distribución de los residuos (H_0 : los residuales siguen una distribución normal); por ejemplo, para el modelo semanal de PM_{2.5} con base KNN se obtuvo un p -valor de 0.071, el cual al ser mayor al nivel de significancia de 0.05 no permite rechazar H_0 , confirmando la normalidad de los residuales al 95% de confianza. Finalmente, se empleó la **Prueba de Ljung-Box** para detectar autocorrelación residual en el primer rezago, obteniendo un estadístico y un valor p asociado (como el caso citado de $Q = 0.02$ y $\text{Prob}(Q) = 0.89$) que no permiten rechazar la hipótesis de ausencia de autocorrelación, lo que demuestra que no queda información predecible remanente en los residuos. Evaluadas de manera conjunta al 95% de confianza, estas tres pruebas sustentan que los modelos ARMA/SARIMA seleccionados cumplen rigurosamente con los requisitos de un buen modelo de pronóstico según la metodología clásica. Aquí se muestra los modelos Box jenkins : 2

Contaminante	Modelo	MAE	RMSE	R^2
Diario				
PM _{2.5}	ARMA(6,0,1)	1.495	2.124	-0.276
PM _{2.5}	ARMA(7,0,1)	1.823	2.288	-0.481
O ₃	AR(1)	5.984	6.391	-0.479
O ₃	ARMA(1,1)	8.952	9.360	-2.173
NO _x	ARMA(4,0,1)	8.756	10.216	0.142
NO _x	ARMA(4,0,3)	9.815	11.499	-0.223
Semanal				
PM _{2.5}	SARMA(2,0,1)(5,0,1,52)	5.110	6.321	-0.226
PM _{2.5}	AR(1)	5.320	6.583	-0.329
O ₃	SARMA(1,0,1)(1,0,0,52)	6.611	6.391	-0.479
O ₃	SARMA(2,2)(1,0,0,52)	6.618	8.368	-1.741
NO _x	SARIMA(3,0,1)(3,0,0,52)	9.396	13.562	-0.011
NO _x	SARMA(3,0,2)(3,0,0,52)	8.670	13.564	-0.011

Tabla 2: Comparativa de métricas de los modelos Box-Jenkins evaluados.

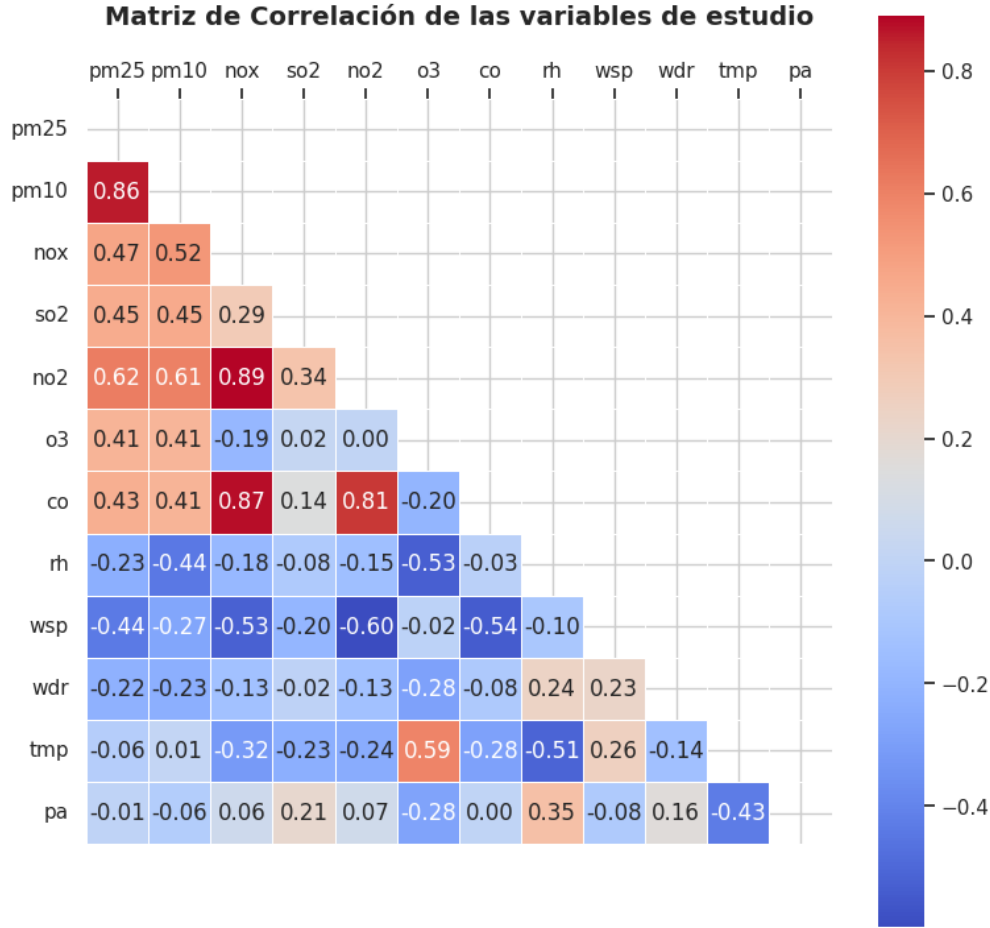


Fig. 1: Matriz de correlación

V MODELADO RANDOM FOREST POR CONTAMINANTE

Para cada contaminante se implementó un modelo de regresión basado en Random Forest utilizando las series temporales diaria y semanal previamente preprocesadas. Como variables predictoras se emplearon los contaminantes atmosféricos PM_{2.5}, PM₁₀, NO_x (Óxidos de nitrógeno), SO₂ (Dióxido de azufre), NO₂ (Dióxido de nitrógeno), O₃ (Ozono), CO (Monóxido de carbono) y las variables meteorológicas RH (Humedad relativa), WSP (Velocidad del viento), WDR (Dirección del viento), TMP (Temperatura) y PA (Presión atmosférica). Con el objetivo de capturar la de-

pendencia temporal de las series, se generaron características mediante una estrategia de ventanas deslizantes. Para la serie diaria se utilizaron rezagos de 1, 3, 7 y 14 periodos, diferencias temporales, medias móviles y tendencias locales. En la serie semanal se emplearon rezagos de 1, 2, 4 y 8 semanas, además de diferencias temporales adaptadas a dicha resolución. El modelo Random Forest Regressor fue implementado utilizando la biblioteca **Scikit-learn** con los siguientes hiperparámetros: 300 árboles de decisión ($n_estimators=300$), profundidad máxima sin restricción ($max_depth=None$), un mínimo de dos muestras por hoja ($min_samples_leaf=2$), semilla aleatoria igual a 42 y procesamiento paralelo mediante todos los núcleos disponibles del sistema

($n_jobs=-1$). Finalmente, el desempeño del modelo se evaluó mediante las métricas MAE, RMSE y R^2 , además del análisis de la importancia de las variables y la comparación entre los valores observados y predichos.

Serie Diaria			
Contaminante	MAE	RMSE	R^2
Sin escalado			
PM _{2.5}	1.634	2.356	0.908
NO _x	5.082	6.613	0.896
O ₃	1.264	1.994	0.964
RobustScaler			
PM _{2.5}	0.203	0.281	0.9178
NO _x	0.144	0.240	0.9426
O ₃	0.066	0.103	0.9857
PowerTransformer			
PM _{2.5}	0.178	0.257	0.903
NO _x	0.118	0.188	0.947
O ₃	0.045	0.070	0.986
Serie Semanal			
Sin escalado			
PM _{2.5}	1.925	2.302	0.820
NO _x	4.071	5.104	0.890
O ₃	2.560	3.485	0.827
RobustScaler			
PM _{2.5}	0.358	0.429	0.798
NO _x	0.293	0.388	0.846
O ₃	0.230	0.294	0.873
PowerTransformer			
PM _{2.5}	0.328	0.418	0.802
NO _x	0.215	0.299	0.859
O ₃	0.156	0.193	0.884

Tabla 3: Comparativa de métricas del modelo Random Forest para las series diaria y semanal según el tipo de escalado.

VI MODELADO XGBOOST REGRESSOR POR CONTAMINANTE

Para cada contaminante (PM_{2.5}, NO_x, O₃) se entrenó un modelo XGBoost mediante `RandomizedSearchCV`, empleando un espacio de búsqueda orientado a reducir la complejidad del modelo y fortalecer su regularización, con

el fin de mitigar el sobreajuste; dicho espacio incluyó rangos acotados para el número de árboles (`n_estimators`: 150–500) y su profundidad (`max_depth`: 2–4), una tasa de aprendizaje baja (`learning_rate`: 0.01–0.08), parámetros de submuestreo aleatorio (`subsample`: 0.4–0.7 y `colsample_bytree`: 0.4–0.8), así como penalizaciones mediante `gamma` (0.1–2), `reg_alpha` (0.1–2.0) y `reg_lambda` (1.5–3.0). Se optó por `RandomizedSearchCV` en lugar de `GridSearchCV` por su mayor eficiencia computacional, evaluando 60 combinaciones aleatorias mediante `TimeSeriesSplit` (5 particiones) para respetar el orden cronológico de los datos, y el entrenamiento y evaluación final se realizaron con una división cronológica 80/20 entre los conjuntos de entrenamiento y prueba, midiendo el desempeño con las métricas MAE, RMSE y R^2 . Los modelos obtenidos se encuentran en la siguiente tabla : 4, ?? Los modelos con mejor desempeño fueron. En frecuencia Diaria: **RobustScaler**: el modelo de O₃ alcanzó el mejor ajuste general ($R^2 = 0.9177$, RMSE = 0.2063), seguido de cerca por PM_{2.5} ($R^2 = 0.9136$, RMSE = 0.2133), mientras que NO_x presentó un desempeño más moderado ($R^2 = 0.7687$, RMSE = 0.4163). En frecuencia semanal, el modelo de PM_{2.5} con escalado **Standard** logró el mayor R^2 del conjunto (0.9288), aunque con un error absoluto considerablemente más alto (MAE = 1.2523) debido a la escala original de la variable; en contraste, los modelos semanales de NO_x y O₃ con **RobustScaler** obtuvieron ajustes más bajos (R^2 de 0.6655 y 0.7457, respectivamente), lo que sugiere que la frecuencia diaria favorece la capacidad predictiva del modelo para estos contaminantes.

Modelos Diarios			
Contaminante	MAE	RMSE	R^2
Standard			
PM _{2.5}	1.5049	1.9831	0.9549
NO _x	2.3610	3.0409	0.9749
O ₃	3.6924	4.7015	0.8501
RobustScaler			
PM _{2.5}	0.1637	0.2133	0.9136
NO _x	0.3152	0.4163	0.7687
O ₃	0.1604	0.2063	0.9177
Normalizado (PowerTransformer)			
PM _{2.5}	0.1837	0.2368	0.9416
NO _x	0.3742	0.4791	0.7683
O ₃	0.2039	0.2626	0.9351
Modelos Semanales			
Standard			
PM _{2.5}	1.2523	1.7707	0.9288
NO _x	1.8103	2.4053	0.9718
O ₃	3.1624	4.0210	0.8249
RobustScaler			
PM _{2.5}	0.2662	0.3520	0.7905
NO _x	0.3493	0.4660	0.6655
O ₃	0.2764	0.3554	0.7457
Normalizado (PowerTransformer)			
PM _{2.5}	0.3547	0.4642	0.7689
NO _x	0.3985	0.5142	0.7315
O ₃	0.3665	0.4688	0.7957

Tabla 4: Comparativa de métricas de los modelos XGBoost diarios y semanales.

VII TABLA RESUMEN FINAL: COMPARATIVA DE MODELOS

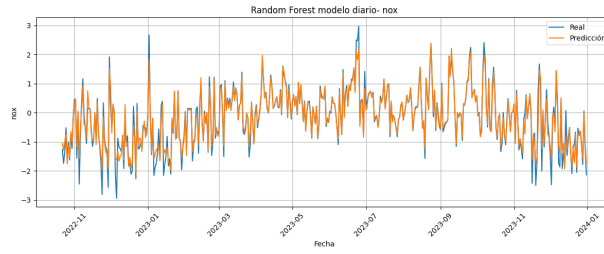
La Tabla 5 resumen consolida los mejores modelos obtenidos. Los resultados de Bos Jenkins evaluada en niveles, Random Forest y Xgboost se denotan en su escala transformada (RobustScaler/PowerTransformer(Normalizado)) por lo que su interpretación directa evalúa únicamente la dinámica interna relativa de la predicción. Además se agregan las gráficas de los óptimos: 2

Contaminante	Modelos Óptimos	MAE	RMSE	R^2
Diario				
PM _{2.5}	Box-Jenkins (KNN)	1.4953	2.1249	-0.2766
	Random Forest (PowerTransformer)	0.178	0.257	0.903
	XGBoost (KNN)	0.1637	0.2133	0.9136
O ₃	Box-Jenkins (KNN)	5.9847	6.3918	-0.4795
	Random Forest (PowerTransformer)	0.045	0.070	0.986
	XGBoost (RobustScaler)	0.1604	0.2063	0.9177
NO _x	Box-Jenkins (KNN)	8.7569	10.2167	0.0342
	Random Forest (PowerTransformer)	0.118	0.188	0.947
	XGBoost (RobustScaler)	0.3152	0.4163	0.7687
Semanal				
PM _{2.5}	Box-Jenkins (KNN)	5.1104	6.3214	-0.2260
	Random Forest (PowerTransformer)	0.328	0.418	0.802
	XGBoost (RobustScaler)	1.2523	1.7707	0.9288
O ₃	Box-Jenkins (KNN)	6.6114	8.3455	-1.7269
	Random Forest (PowerTransformer)	0.156	0.193	0.884
	XGBoost (RobustScaler)	0.2764	0.3554	0.7457
NO _x	Box-Jenkins (KNN)	9.3966	13.5620	-0.0112
	Random Forest (PowerTransformer)	0.215	0.299	0.859
	XGBoost (RobustScaler)	0.3493	0.4660	0.6655

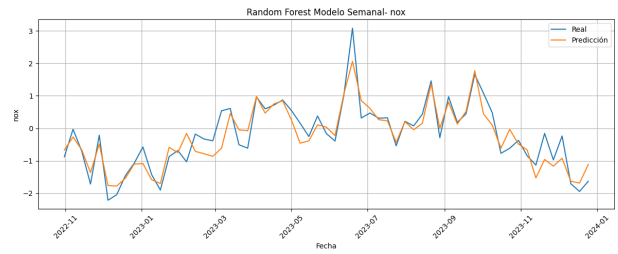
Tabla 5: Comparativa final de modelos seleccionados por contaminante (diario y semanal).

VIII DISCUSIÓN E INTERPRETACIÓN

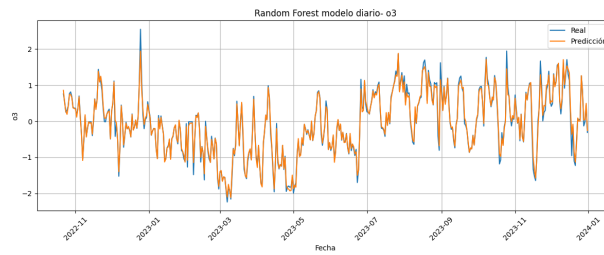
Los modelos tradicionales de series de tiempo logran aproximaciones válidas en términos de ruido blanco y estacionariedad. No obstante, las dinámicas de contaminantes secundarios o reactivos como el O₃ y NO_x están fuertemente influenciadas por interacciones fotoquímicas y meteorológicas no lineales externas. Es bajo estas condiciones donde los algoritmos basados en árboles como XGBoost demuestran ventajas operativas al reducir significativamente los errores de estimación mediante la incorporación simultánea de variables exógenas concurrentes. Se debe acotar que la comparación numérica mantiene un carácter orientativo debido a las diferencias en los esquemas de validación (cronológica fuera de muestra en Box-Jenkins y partición controlada por rezagos en XGBoost).



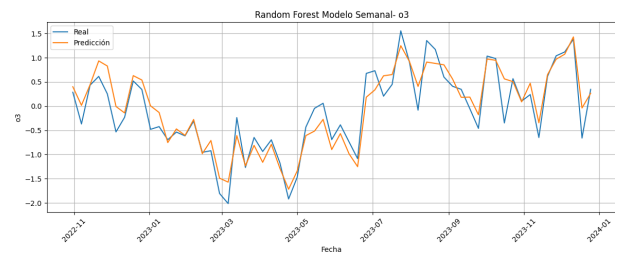
(a) Modelo Diario NO_X



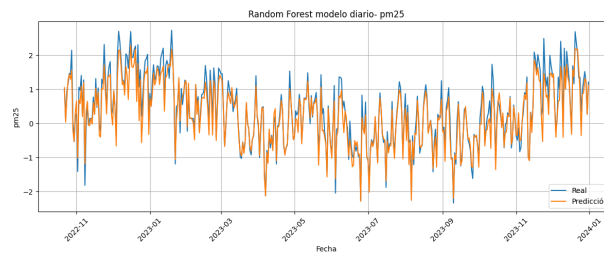
(b) Modelo Semanal NO_X



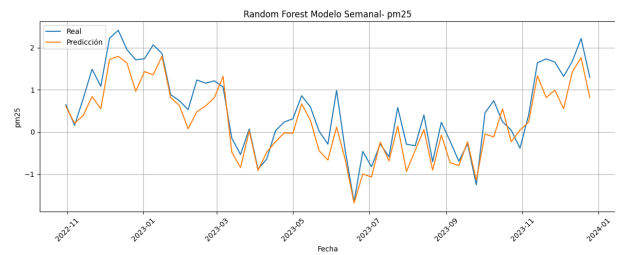
(c) Modelo Diario O_3



(d) Modelo Semanal O_3



(e) Modelo Diario $PM_{2.5}$



(f) Modelo semanal $PM_{2.5}$

Fig. 2: Gráficas de Pronósticos NO_x , O_3 y $PM_{2.5}$.

IX CONCLUSIONES

1. La técnica de imputación KNN supera de forma sistemática a los esquemas lineales univariados para reconstruir series atmosféricas.
2. La metodología clásica de Box–Jenkins provee modelos linealmente robustos que satisfacen rigurosamente los supuestos de normalidad y diagnóstico residual.
3. XGBoost demuestra una alta capacidad predictiva para modelar contaminantes con dinámicas gobernadas por agentes externos (NO_x y O_3).
4. El análisis multivariado mediante Random Forest identificó una fuerte dependencia física no lineal (71-74%) entre la concentración de partículas finas $\text{PM}_{2.5}$ y las fluctuaciones locales de la presión atmosférica en la Ciudad de México.

AGRADECIMIENTOS

Se agradece al Programa Delfín por el respaldo y la oportunidad otorgada para la realización de esta estancia de investigación. Asimismo, se reconoce a la Escuela Superior de Física y Matemáticas (ESFM) por el apoyo institucional brindado.

De igual manera, se agradece el soporte técnico y la información meteorológica e histórica provista por la red de monitoreo de la estación Merced para el desarrollo de los entornos experimentales evaluados.

REFERENCIAS

- [1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis:*

Forecasting and Control, 5th ed. New Jersey: Wiley, 2015.

- [2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.
- [3] S. Seabold and J. Perktold, «Statsmodels: Econometric and statistical modeling with Python,» in *Proc. 9th Python in Science Conf.*, 2010.
- [4] T. G. Smith et al., «pmdarima: ARIMA estimators for Python,» 2017. [Online]. Available: <https://alkaline-ml.com/pmdarima/>
- [5] T. Chen and C. Guestrin, «XGBoost: A Scalable Tree Boosting System,» in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [6] L. Breiman, «Random Forests,» *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] F. Pedregosa et al., «Scikit-learn: Machine Learning in Python,» *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] G. E. A. P. A. Batista and M. C. Monard, «An analysis of four missing data treatment methods for supervised learning,» *Applied Artificial Intelligence*, vol. 17, no. 5-6, pp. 519–533, 2003.
- [9] M. Lepot, J.-B. Aubin, and F. H. L. R. Clemens, «Interpolation in time series: An introductive overview of existing methods, their performance criteria and uncertainty assessment,» *Water*, vol. 9, no. 10, p. 796, Oct. 2017.
- [10] GeeksforGeeks, «How to deal with missing values in a time-series in Python,» 2024. [Online]. Available: <https://www.geeksforgeeks.org/data-science/how-to-deal-with-missing-values-in-a-timeseries-in-python/> [Accessed: Jul. 16, 2026].

- [11] DataCamp. *RMSE: Root Mean Square Error* [Online]. Available: <https://www.datacamp.com/de/tutorial/rmse>. [Accessed: Jul. 16, 2026].
- [12] IABigData, «Error medio cuadrado (ECM/MSE),» 2024. [Online]. Available: <https://iabigdata-soka-4ae9e223e32444ac5ae3d78afbd55fd9aa6da1c19d9679bf.gitlab.io/post/2024-09-09-pia-metrics/> [Accessed: Jul. 16, 2026].
- [13] GeeksforGeeks. (2025). *PowerTransformer in scikit-learn* [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/powertransformer-in-scikit-learn/>. [Accessed: Jul. 16, 2026].
- [14] F. Pedregosa et al. *RobustScaler, scikit-learn-documentation* [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>. [Accessed: Jul. 16, 2026].
- [15] Complex Systems AI. *Validación cruzada* [Online]. Available: <https://complex-systems-ai.com/es/pronostico-de-prediccion/validacion-cruzada/>. [Accessed: Jul. 16, 2026].
- [16] E. Kavlakoglu and E. Russi. (2024, May 9). *¿Qué es XGBoost?* [Online]. IBM. Available: <https://www.ibm.com/mx-es/think/topics/xgboost>. [Accessed: Jul. 16, 2026].
- [17] APMonitor. *XGBoost Regressor* [Online]. Available: <https://apmonitor.com/pds/index.php/Main/XGBoostRegressor>. [Accessed: Jul. 16, 2026].
- [18] XGBoost Developers. *XGBoost Parameters* [Online]. Available: <https://xgboost.readthedocs.io/en/stable/parameter.html>. [Accessed: Jul. 16, 2026].
- [19] F. Pedregosa et al. *RandomizedSearchCV — scikit-learn documentation* [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html. [Accessed: Jul. 16, 2026].